

RooTitanium — Impostazioni: sezioni Password e Permessi

Guida di consegna passo-passo per Claude Opus (continuazione del lavoro di Fable)

Progetto: mini-browser QtWebEngine per SailfishOS 5.1 · Engine: qt6-qtwebengine 6.8.3 (Chromium 122) · 13 luglio 2026

In due righe. Le due voci “Password” e “Permessi” oggi sono solo segnaposto “In arrivo” nella pagina Impostazioni. **Permessi** si fa interamente in QML con l'API nativa di WebEngine 6.8 (nessuna ricompilazione). **Password** invece **non** ha alcuna API in QtWebEngine: va costruito a mano e richiede una modifica C++ a `main.cpp` con relativa ricompilazione del binario. Fai **prima Permessi** (veloce e a basso rischio), poi Password.

0. Contesto minimo dell'architettura

Ti serve sapere solo questo per lavorare senza sorprese. Tutto il browser vive in un unico file QML.

File	Ruolo
<code>smoke-test/test.qml</code>	Tutta la UI e la logica del browser (~1700 righe). Qui vanno quasi tutte le modifiche.
<code>smoke-test/main.cpp</code>	Helper C++ <code>rtNative</code> esposto al QML (DBus, filesystem). Si tocca solo per la parte Password.
<code>smoke-test/run.sh</code>	Avvio sul device (env Chromium, flag, DevTools su 9222).

Ciclo di lavoro sul device

Il device di test è `defaultuser@192.168.1.119`. Il bundle sta in `~/webengine-smoke/`. Per provare una modifica al solo QML:

```
scp smoke-test/test.qml defaultuser@192.168.1.119:/home/defaultuser/webengine-smoke/test.qml

# riavvio: MAI "pkill -f webengine-smoke" via ssh (ucciderebbe la shell). Kill via /proc:

ssh defaultuser@192.168.1.119 '

    for p in /proc/[0-9]*; do case "$(readlink $p/exe 2>/dev/null)" in */webengine-smoke) kill
    ${p#/proc/};; esac; done

    sleep 2; nohup /home/defaultuser/webengine-smoke/run.sh >/dev/null 2>&1 &'
```

Il log runtime è su device in `/tmp/rootitanium.log`. I `console.log` QML ci arrivano (`run.sh` forza `QT_FORCE_STDERR_LOGGING=1`). Per il debug delle pagine: `tunnel ssh -L 9223:localhost:9222 ...` poi CDP su `http://localhost:9223`.

Persistenza impostazioni (già pronta)

C'è una tabellina chiave/valore in SQLite (LocalStorage QML). Usa queste funzioni già esistenti in `test.qml`:

- `kvGet(chiave, default) / kvSet(chiave, valore)` — lettura/scrittura grezza.
- `loadCfg()` — chiamata all'avvio; qui aggiungi il caricamento di eventuali nuove preferenze.
- `applySetting(k, v)` — punto unico dove un toggle/radio applica e persiste una preferenza.

Come è fatta la pagina Impostazioni

È una **pagina HTML interna** generata da `settingsHtml()` e caricata con `loadInternal(view, "settings", settingsHtml(), "https://settings.local/")`. Non è QML: è HTML puro (deve funzionare anche con JavaScript disattivato). L'interazione avviene con **link-sentinella**: un click su `https://settings.local/...` viene intercettato in `onNavigationRequested` della `WebEngineView` (ramo `u.indexOf("https://settings.local/")===0`), l'azione viene eseguita in QML e la pagina viene rigenerata. Righe, toggle e radio si costruiscono con gli helper `sToggle(k, on, titolo, desc)` già presenti. I due segnaposto da sostituire sono queste righe dentro `settingsHtml()`:

```
<div class="srow dis"><span class="sbody"><span class="st">Password</span></span><span class="badge">In arrivo</span></div>

<div class="srow dis"><span class="sbody"><span class="st">Permessi</span></span><span class="badge">In arrivo</span></div>
```

Modello consigliato (come per Cronologia/Download/Segnalibri): ognuna diventa una riga cliccabile che apre una **propria sotto-pagina interna** (`passwords.local / permissions.local`) con l'elenco e le azioni. Registra i nuovi host `.local` nello stesso `onNavigationRequested`, accanto a quelli esistenti.

1. Sezione PERMESSI (QML puro, nessuna ricompilazione)

QtWebEngine 6.8 espone un'API permessi completa. Oggi in RooTitanium **non c'è alcun gestore**: le richieste dei siti (geolocalizzazione, notifiche, fotocamera, microfono, appunti...) vengono **rifiutate in silenzio**. Vanno fatte due cose: (a) gestire le richieste con un dialogo, (b) mostrarle/revocarle in Impostazioni.

API disponibile (verificata nei sorgenti 6.8.3)

Elemento	Dove	Uso
----------	------	-----

<code>permissionRequested(QWebEnginePermission)</code>	segnale di <code>WebEngineView</code> (REV 6,8)	Sostituisce il deprecato <code>featurePermissionRequested</code> . Arriva a ogni richiesta del sito.
<code>QWebEnginePermission</code>	oggetto passato dal segnale	Proprietà: <code>origin</code> , <code>permissionType</code> , <code>state</code> . Metodi: <code>grant()</code> , <code>deny()</code> , <code>reset()</code> .
<code>profile.persistentPermissionsPolicy</code>	proprietà di <code>WebEngineProfile</code>	<code>AskEveryTime</code> / <code>StoreInMemory</code> / <code>StoreOnDisk</code> . Metti <code>StoreOnDisk</code> per ricordare le scelte.
<code>profile.listAllPermissions()</code>	metodo di <code>WebEngineProfile</code>	Ritorna la lista di tutti i permessi decisi (per la pagina di gestione). Anche <code>listPermissionsForOrigin(url)</code> .

`PermissionType`: `MediaAudioCapture`, `MediaVideoCapture`, `MediaAudioVideoCapture`, `DesktopVideoCapture`, `DesktopAudioVideoCapture`, `MouseLock`, `Notifications`, `Geolocation`, `ClipboardReadWrite`, `LocalFontsAccess`. Stati: `Ask`, `Granted`, `Denied`, `Invalid`.

Passo 1 — Policy persistente sui due profili

In `test.qml`, sui blocchi `WebEngineProfile { id: normalProfile }` e `incognitoProfile` aggiungi (per l'incognito ha senso lasciare `StoreInMemory`, così non resta traccia su disco):

```
// normalProfile

persistentPermissionsPolicy: WebEngineProfile.StoreOnDisk

// incognitoProfile

persistentPermissionsPolicy: WebEngineProfile.StoreInMemory
```

Passo 2 — Gestire la richiesta con un dialogo QML

Nella `WebEngineView` del `Repeater` (dove ci sono già `onJavaScriptDialogRequested`, `onFullScreenRequested`, ecc.) aggiungi il gestore. Riusa lo stile del dialogo già esistente `jsDlg` (o creane uno gemello `permDlg`): mostra origine + tipo tradotto, con “Consenti” / “Blocca”.

```
onPermissionRequested: function(permission) {

    win.showPermission(permission)    // vedi funzione sotto

}
```

In `win` (oggetto radice) aggiungi una funzione che memorizza il permesso pendente e apre il dialogo.

Attenzione: `QWebEnginePermission` va tenuto vivo finché l'utente non decide (salvalo in una `property` `var permPending`), poi chiama `permPending.grant()` o `.deny()`.

```
property var permPending: null

function permLabel(t) {

    // mappa PermissionType -> italiano (usa i valori enum di
    QWebEnginePermission.PermissionType)

    var T = QWebEnginePermission.PermissionType

    if (t === T.Geolocation)      return "Conoscere la tua posizione"
    if (t === T.Notifications)    return "Inviarti notifiche"
    if (t === T.MediaAudioCapture) return "Usare il microfono"
    if (t === T.MediaVideoCapture) return "Usare la fotocamera"
    if (t === T.MediaAudioVideoCapture) return "Usare fotocamera e microfono"
    if (t === T.ClipboardReadWrite) return "Leggere gli appunti"
    if (t === T.LocalFontsAccess)  return "Elencare i caratteri installati"
    if (t === T.MouseLock)         return "Bloccare il puntatore"

    return "Un permesso"
}

function showPermission(p) {

    permPending = p

    permDlg.host = (" " + p.origin).replace(/^[a-z]+:\//i, "").replace(/\/.*$/, "")

    permDlg.msg = permLabel(p.permissionType)

    permDlg.open = true
}

function permDecide(grant) {

    permDlg.open = false

    if (permPending) { if (grant) permPending.grant(); else permPending.deny(); permPending
    = null }
}
}
```

Nota: `QWebEnginePermission` è un tipo QML già importato con `import QtWebEngine`; l'enum si riferenzia come `QWebEnginePermission.PermissionType.Geolocation`. Se il runtime non lo esponesse come atteso, in fallback confronta l'intero (Geolocation = 8, Notifications = 7 nell'ordine dell'enum).

Passo 3 — Sotto-pagina di gestione `permissions.local`

La riga “Permessi” in `settingsHtml()` diventa un link a `https://permissions.local/`. Crea `permissionsHtml()` sul modello di `downloadsHtml()`: elenca `normalProfile.listAllPermissions()`, raggruppa per origin, e per ogni permesso mostra tipo tradotto + stato (Consentito/Bloccato) con un’azione di revoca (link-sentinella che chiama `reset()` o inverte `grant()/deny()`).

```
// dentro onNavigationRequested, accanto ai rami esistenti:

} else if (u.indexOf("https://permissions.local/") === 0) {

    request.action = WebEngineNavigationRequest.IgnoreRequest

    var mR = u.match(/^https:\/\/permissions\.local\/reset\?o=(.*)&t=(\d+)$/ )

    if (mR) {

        var perm = normalProfile.queryPermission(decodeURIComponent(mR[1]), parseInt(mR[2]))

        if (perm.isValid) perm.reset()

    }

    win.loadInternal(this, "permissions", win.permissionsHtml(),
"https://permissions.local/")

}
```

Ricorda di marcare `permissions.local` come pagina interna nella logica già presente (le funzioni che escludono gli host `*.local` da cronologia/condivisione/PDF lo fanno già via regex: nessuna aggiunta necessaria).

Caveat backend SailfishOS. Concedere il permesso è una cosa; ottenere il dato è un'altra. Geolocalizzazione, fotocamera e microfono dipendono dai backend del device (geoclue, pipeline gstreamer/hybris) che in questo bundle non-embedded potrebbero non essere collegati. La **logica dei permessi** (chiedere, ricordare, revocare) è comunque corretta e testabile a sé: `Notifications` e `ClipboardReadWrite` sono i più facili da verificare senza hardware. Non promettere all'utente che la posizione GPS funzioni finché non l'hai vista arrivare.

Test (via CDP)

Per far scattare una richiesta senza aspettare un sito reale, dopo il deploy naviga su una pagina che chiede il permesso, es. `Notification.requestPermission()` o `navigator.geolocation.getCurrentPosition(...)` iniettato via CDP, e verifica che compaia il dialogo. Poi controlla che `listAllPermissions()` mostri la scelta nella pagina di gestione e che sopravviva al riavvio dell'app.

2. Sezione PASSWORD (richiede C++ + compilazione)

Verità architetturale, da leggere prima di scrivere codice. QtWebEngine **non espone alcun password manager al QML**. Il gestore password interno di Chromium esiste ma è sepolto: l'unico aggancio nei sorgenti è `AutofillPopupController` in header privati (`..._p.p.h`), non istanziabile da QML. Quindi "salvare le password" qui significa **costruire un mini gestore nostro**: catturare le credenziali dai form, salvarle cifrate, e reinserirle al caricamento. Non è un toggle: è una funzionalità a sé.

Meccanismo di trasporto JS → app (la decisione chiave)

Per catturare user/password dal form serve che il JavaScript della pagina comunichi all'app. I canali possibili:

Canale	Giudizio
Navigazione a sentinella <code>*.local</code> (come per le altre pagine)	NO: navigare via <code>location.href</code> sul submit abortisce il login. Da escludere.
Polling <code>runJavaScript</code> dal QML	Fragile per catturare l'istante del submit; usalo solo per l'autofill (lettura/scrittura campi), non per la cattura.
Custom URL scheme handler in C++ (<code>QWebEngineUrlSchemeHandler</code>), chiamato da JS con <code>fetch("rt-cred:save?...")</code>	Sì, consigliato. Un fetch verso uno schema custom viene intercettato in C++ senza bloccare la navigazione del form. Canale JS → C++ pulito e unidirezionale.
<code>QWebChannel</code> (<code>qwebchannel.js</code> + oggetto registrato)	Alternativa standard e bidirezionale, ma più infrastruttura da montare. Valida se preferisci non toccare gli URL scheme.

Consiglio: **URL scheme handler** per la cattura (JS → C++) e **runJavaScript** per l'autofill (C++/QML → JS). Sotto trovi i passi in quest'ordine.

Passo 1 — Registrare lo schema e lo store in `main.cpp`

In `main.cpp`, prima di creare l'engine, registra lo schema (deve avvenire **prima** di `QtWebEngineQuick::initialize()`):

```
QWebEngineUrlScheme scheme("rt-cred");

scheme.setSyntax(QWebEngineUrlScheme::Syntax::Path);

scheme.setFlags(QWebEngineUrlScheme::SecureScheme |
QWebEngineUrlScheme::LocalAccessAllowed);
```

```
QWebEngineUrlScheme::registerScheme(scheme);
```

Poi installa un `QWebEngineUrlSchemeHandler` sui due profili (via `profile->installUrlSchemeHandler("rt-cred", handler)`). Il handler estrae i parametri dalla query, li passa allo store, e risponde con un 204/blob vuoto. Metti store e cifratura dentro `rtNative` (già esposto al QML) oppure in una classe gemella: così il QML può leggere l'elenco per la pagina di gestione.

Passo 2 — Schema SQLite

Puoi riusare lo stesso DB del resto (tabella nuova) oppure un file dedicato. Campi minimi:

```
CREATE TABLE IF NOT EXISTS passwords(  
    origin    TEXT NOT NULL,      -- es. https://x.com  
    username TEXT NOT NULL,  
    password BLOB NOT NULL,      -- CIFRATA, mai in chiaro  
    ts        INTEGER,  
    PRIMARY KEY(origin, username)  
);
```

Passo 3 — Cattura credenziali (user script)

Aggiungi uno script iniettato (come `rtDnt` / `rtYtTapFix` in `buildScripts()`) che sull'evento `submit` di qualunque form individua i campi password e il probabile username, e li invia:

```
document.addEventListener('submit', function(e){  
    var f = e.target; if (!f || !f.querySelector) return;  
    var pw = f.querySelector('input[type=password]'); if (!pw || !pw.value) return;  
    // username: il primo input testuale/email prima della password (euristica)  
    var user = '';  
    var inputs = f.querySelectorAll('input');  
    for (var i=0;i<inputs.length;i++){ var t=inputs[i]; if (t===pw) break;  
        if (/text|email|tel/.test(t.type)) user = t.value; }  
    var q = 'o='+encodeURIComponent(location.origin)+'&u='+encodeURIComponent(user)  
    + '&p='+encodeURIComponent(pw.value);  
    fetch('rt-cred:save?' + q).catch(function({})); // intercettato in C++, non blocca il  
    submit
```

```
}, true);
```

Euristiche da rifinire: form multi-step (username e password in pagine separate, come proprio X), campi `autocomplete="new-password"` (registrazione, non da salvare come login), più form nella pagina. Per il primo taglio va bene la versione semplice; annota i limiti.

Passo 4 — Cifratura a riposo

Onestà sulla sicurezza. Salvare password è dato sensibile. Il minimo accettabile è cifrarle a riposo (non in chiaro nel DB). Con solo Qt a disposizione, la via pragmatica è una chiave derivata da un segreto del device (es. machine-id) usata con un cifrario simmetrico. **Ma** ciò protegge solo da chi legge il file, non da chi esegue codice come l'utente. La soluzione robusta su SailfishOS è il framework *Sailfish Secrets* o una **master password** chiesta all'utente (chiave mai su disco). Decidi il livello con l'utente prima di promettere "sicuro"; per un primo taglio dichiara esplicitamente che è "offuscamento locale", non cassaforte.

Passo 5 — Autofill al caricamento

Su `onLoadingChanged` con `LoadSucceededStatus` (dove già si registra la cronologia), interroga lo store per `origin` corrente; se c'è una credenziale, iniettila con `runJavaScript` riempiendo i campi (senza inviare il form — l'utente conferma). Offri anche un banner "Salvare la password?" la prima volta (può essere un toast QML dopo la cattura, con "Salva"/"No").

Passo 6 — Sotto-pagina di gestione passwords.local

Come per i Permessi: la riga "Password" apre `https://passwords.local/`; `passwordsHtml()` elenca origine + username (**mai** mostrare la password; al più un pulsante "mostra" dietro conferma), con azione elimina (sentinella che cancella la riga). Modella su `downloadsHtml()`.

Passo 7 — Ricompilare il binario

Toccando `main.cpp` devi ricostruire `webengine-smoke` (il solo `scp` del QML non basta). Il procedimento è già documentato in memoria (progetto): `moc` del target + `g++` dentro l'engine `sb2`.

```
sfdk engine exec sb2 -t SailfishOS-5.1.0.11-aarch64.default

# moc: /usr/lib64/qt6/libexec/moc su main.cpp ; poi g++ con

# -I /usr/include/qt6/... e gli include WebEngine dal build tree:

# packaging/qt6-qtwebengine-sfos/build/BUILD/qt6-qtwebengine-6.8.3/upstream/include

# -L scratch/webengine-bundle/lib
```

Verifica la disponibilità degli header `QWebEngineUrlScheme` / `QWebEngineUrlSchemeHandler` nell'include tree prima di iniziare; sono in `QtWebEngineCore`.

3. Ordine consigliato e checklist

1. **Permessi prima** — tutto QML, nessuna ricompilazione, verificabile subito. Chiudi questa per intero.
2. Poi **Password**, e prima di scrivere codice **concorda con l'utente il livello di sicurezza** (offuscamento vs master password vs Sailfish Secrets).
3. Registra schema URL e handler in `main.cpp`, ricompila, verifica il canale `rt-cred:` con un `fetch` di prova.
4. Cattura → store cifrato → autofill → pagina di gestione, in quest'ordine, testando ogni pezzo via CDP.
5. Aggiorna la memoria di progetto e committa (mai push remoto: il repo è privato, il push si fa con la skill dedicata).

Nota sul bug login X (contesto, non parte di questo lavoro). Fable ha già corretto due cause probabili: il profilo che girava off-the-record (i cookie non persistevano) e un fingerprint incoerente (User-Agent vs Client Hints vs Accept-Language). Il login X **non** è ancora risolto. La sezione Password qui sopra **non c'entra** con quel bug: costruirla non lo risolverà. Tienili separati.